

User Guide – LLM Observability & Evaluation Platform

Table of Contents

1. [Introduction](#)
2. [Key Concepts](#)
 - 2.1. [Organizations](#)
 - 2.2. [Projects](#)
 - 2.3. [Applications](#)
 - 2.4. [Traces](#)
 - 2.5. [Observations](#)
 - 2.6. [Metrics](#)
 - 2.7. [Auto Eval](#)
 - 2.8. [Model Hub](#)
 - 2.9. [Metric Hub](#)
3. [Getting Started](#)
 - 3.1. [Create an Account](#)
 - 3.2. [Create an Organization](#)
 - 3.3. [Create a Project](#)
 - 3.4. [Create an Application](#)
4. [Instrumentation \(Integrating Observix with your Application\)](#)
 - 4.1. [Integration with LLM application](#)
5. [Verify the trace in the platform \(Dashboard\)](#)
6. [First Trace Walkthrough \(Traces\)](#)
 - 6.1. [Evaluate Node](#)
 - 6.2. [Evaluate Trace](#)
7. [Model Hub](#)
 - 7.1. [Registering a Model](#)
8. [Metric Hub](#)
 - 8.1. [Metrics](#)
 - 8.2. [Metric Playground](#)
9. [Evaluations](#)
 - 9.1. [Evaluation Dashboard](#)
 - 9.2. [Evaluation Runs](#)
 - 9.3. [Auto Evaluation](#)

1. Introduction

Welcome to the **LLM Observability & Evaluation Platform**. This platform is designed to provide comprehensive monitoring and quality assurance for your agentic and LLM-powered workflows.

The platform helps you to:

- Track, monitor, and evaluate agentic workflows and agent executions.
- Log prompts, responses, tool calls, and metadata.
- Evaluate agent performance using automated metrics.
- Debug failures and reduce hallucinations.
- Analyze cost, latency, and quality trends.

This guide explains how to install, configure, and use the platform effectively.

2. Key Concepts

2.1 Organizations

An **Organization** is the highest-level workspace that groups projects. It provides centralized access control, usage tracking, and configuration management across all projects in the platform.

2.2 Projects

A Project is a workspace within an Organization that groups one or more Applications. Each Project contains all observability data including traces, evaluations, and metrics generated by its Applications. Projects help organize and isolate data by application, environment, or use case while enabling centralized monitoring and analysis.

2.3 Applications

An Application is a unit within a Project that represents a specific LLM-powered service, agent, or workflow. Each Application generates its own traces, evaluations, and metrics, enabling focused monitoring, debugging, and performance analysis for individual use cases within the Project.

2.4 Traces

A Trace represents a single end-to-end execution of an Application and is composed of multiple Observations. It captures the following:

- Multiple LLM calls
- Tool invocations
- Agent decisions
- Intermediate steps

2.5 Observations

An Observation is a single recorded event within a Trace, such as an LLM call, a tool invocation, or an intermediate processing step. Observations store structured data including inputs, outputs, metadata, and timing information, providing granular visibility into each step of an Application's execution.

2.6 Metrics

Predefined or custom metrics used for the evaluation of the LLM responses, agentic applications, RAG applications

Examples:

- Tool selection accuracy - Agentic Evaluation
- Answer Relevancy - RAG evaluation
- Hallucination detection - LLM output evaluation
- Agent Routing - Agentic Evaluation

2.7 Auto Eval

Auto Evaluation allows users to configure which evaluation metrics are automatically executed for every tracked Trace. By defining metric rules at the Project or Application level, the platform runs selected evaluations on each Trace in real time, ensuring continuous quality monitoring without manual intervention.

2.8 Model Hub

The Model Hub is a centralized registry where users can register and manage their LLM models for use in evaluations. Registered models can be selected as evaluators for metrics such as relevance, hallucination detection, and quality scoring, enabling consistent and configurable evaluation across Projects and Applications.

2.9 Metric Hub

The Metric Hub is a centralized catalog of evaluation metrics that provides metric definitions, usage guidelines, and an interactive playground for testing and tuning metrics. It enables users

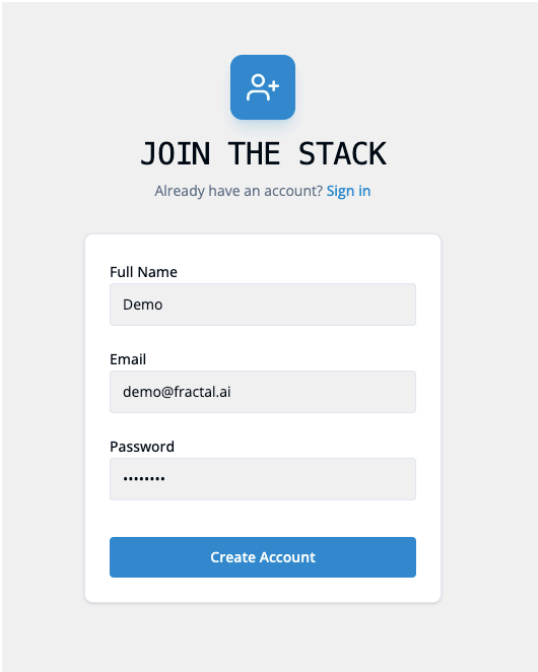
to explore available metrics, understand their behavior, and validate evaluation logic before applying them to Traces and Auto Evaluation workflows.

3. Getting Started

3.1 Create an Account

To get started, create an account on the platform using the signup page.

- Open the signup URL: `https://<domain>/signup`



- After successful registration, you will be redirected to the login page: `https://<domain>/login`
- Log in using the credentials you created during signup.
- Upon successful login, you will be taken to the dashboard where you can create your first Organization and Project.

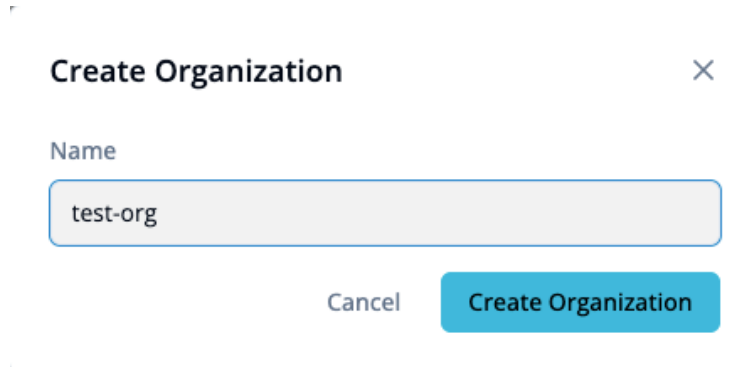
3.2 Create an Organization

From the dashboard, click on Organization → Create Organization. A modal window will appear.

- Enter the Organization name.

- Click Create to finalize the Organization.

Once created, the Organization becomes your top-level workspace for managing Projects, Applications



A modal window titled "Create Organization" with a close button (X) in the top right corner. Below the title is a label "Name" followed by a text input field containing "test-org". At the bottom of the modal are two buttons: "Cancel" and "Create Organization".

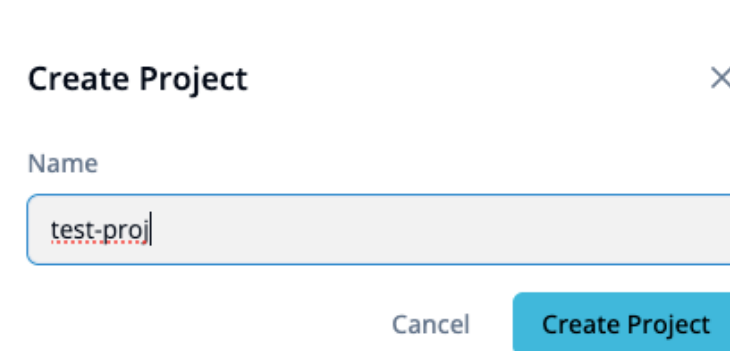
3.3 Create a Project

After creating an Organization, you can create a Project to start tracking your Applications and their data.

From the dashboard, navigate to Project → Create Project. A modal window will appear.

- Enter a name for the Project.
- Click Create to finalize the Project.

Once created, the Project will act as a workspace that groups multiple Applications and their associated Traces, Observations, and Evaluation metrics.



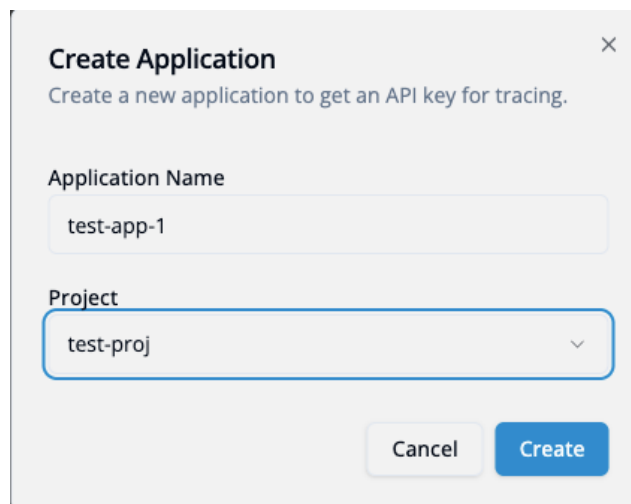
A modal window titled "Create Project" with a close button (X) in the top right corner. Below the title is a label "Name" followed by a text input field containing "test-proj". At the bottom of the modal are two buttons: "Cancel" and "Create Project".

Note: *It is recommended to create separate Projects for different applications or environments (for example, dev and production).*

3.4 Create an Application

After creating a Project, you can create an Application to begin tracking traces and evaluations for a specific workflow or use case.

- Navigate to the <https://<domain>/application> page from the dashboard.
- Click **New Application**. A modal window will appear.
- Enter a name for the Application and choose the project under which you want to create the application
- Click Create to finalize.



The image shows a modal window titled "Create Application" with a close button (X) in the top right corner. Below the title is a subtitle: "Create a new application to get an API key for tracing." There are two input fields: "Application Name" with the value "test-app-1" and "Project" with a dropdown menu showing "test-proj". At the bottom right are two buttons: "Cancel" and "Create".

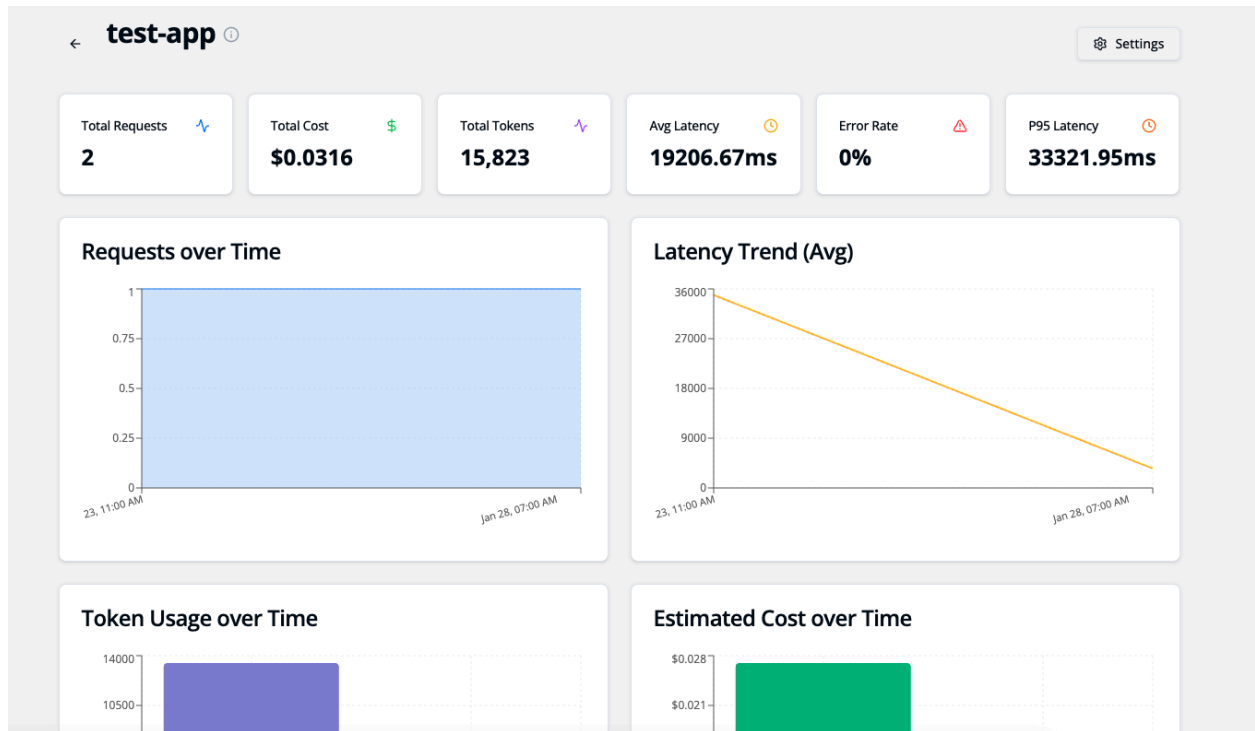
Once the Application is created, an API key is automatically generated and assigned to it. Copy this API key and store it securely, as it is required to send Traces and Observations from your application to the platform.



The image shows the "Applications" dashboard. At the top, there's a header "Applications" with a help icon and a "+ New Application" button. Below the header, it says "Showing applications for project: test-proj". A green notification box states "API Key Created!" and "Copy this key now. You won't be able to see it again." with a "Copy Key" button. The API key is displayed as "sk-ntBSL0fQJt1x71PV8PH5WusVSRsKu2rYCp6Jj fgMeL8". Below the notification is a table with columns: Name, Project, API Key, and Actions.

Name	Project	API Key	Actions
test-app	test-proj	sk-29mdj****	
test-app-1	test-proj	sk-ntBSL****	

Each application has its own dashboard to track the model usage, latency, requests, failure



In the settings, we can change the name of the application and modify the rubrics prompt which will be used for the agentic evaluations

Application Settings

Update application details and configuration.

Application Name

Rubric Prompt

Define the scoring guidelines or rubric for evaluations associated with this application.

CONSISTENT SCORING GUIDELINES:

Score Distribution:

- 90-100 (Exceptional): Top 10% performance, benchmark quality
- 70-80 (Proficient): Top 25% performance, production ready
- 50-60 (Adequate): Meets basic requirements, needs optimization
- 30-40 (Below Standard): Significant issues, requires improvement
- 10-20 (Unacceptable): Critical failures, not functional

CancelSave Changes

4. Instrumentation (Integrating Observix with your Application)

4.1 Integration with LLM application

Once your Application is created and an API key is generated, you can instrument your agentic application to start sending Traces and Observations to the platform.

Step 1: Configure Environment Variables

Set the following environment variables in your agentic application:

```
OBSERVIX_API_KEY=<your_api_key>
OBSERVIX_HOST=<observix_domain>
```

These variables authenticate your application and define the Observix backend endpoint.

Step 2: Clone the Observix Instrumentation Submodule

In your agentic application repository, clone the Observix instrumentation submodule hosted at:

```
https://gitlab.fractal.ai/arash.arora/observix
```

Example:

```
git submodule add https://gitlab.fractal.ai/arash.arora/observix observix
git submodule update --init --recursive
```

This adds the Observix SDK as a submodule to your project.

Step 3: Instrument Agents and Tools

To instrument any agent or tool, import the **observe** decorator and wrap your functions.

```
from observix import observe

@observe(name="agent1", as_agent=True)
def agent1():
    pass
```

```
@observe(name="tool1", as_tool=True)
def tool1(): pass
```

Each decorated function call will automatically instrument Observations and group them into Traces for end-to-end visibility.

Step 4: Use Built-in LLM Integrations (Optional)

Observix provides native LLM wrappers for popular providers with automatic instrumentation.

```
from observix.llm.langchain import ChatGroq
from observix.llm.openai import OpenAI, AzureOpenAI, AsyncOpenAI,
AsyncAzureOpenAI
```

You can instantiate LLMs as follows:

```
llm = ChatGroq(model="llama-3.3-70b-versatile")

llm = OpenAI(model="gpt-4o", temperature=0)
```

Note: You must provide the required API keys for the respective LLM providers (such as OpenAI, Azure, or Groq) in your environment variables for these models to function correctly.

Step 5: Run Your Application

Execute your agentic application as usual

- Traces and Observations will automatically appear in the Observix dashboard
- Auto Evaluation metrics (if configured) will run on each Trace

5. Verify the trace in the platform (Dashboard)

After instrumenting your application and running it, you can verify that data is being successfully captured in the dashboard.

- Log in to the platform and navigate to your Project and Application.
- Click on the **Overview** from the left navigation panel.

The Dashboard provides a high-level summary of your Project's activity, usage, and evaluation performance through key metrics and visualizations.

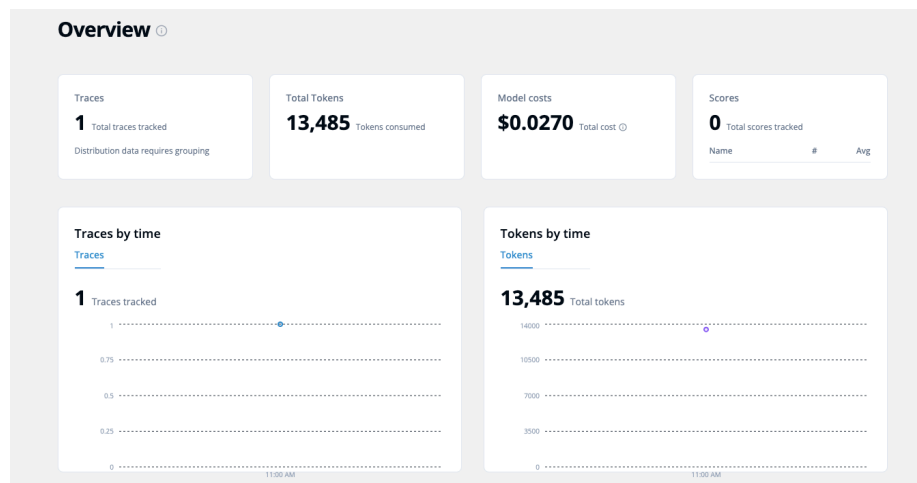
Key Metrics and Graphs:

- **Total Traces:** Number of end-to-end executions tracked.
- **Total Tokens:** Cumulative token consumption.
- **Model Costs:** Estimated cost incurred from model usage.
- **Scores:** Total number of evaluation scores generated.
- **Traces by Time:** Graph showing Trace generation over time.
- **Tokens by Time:** Graph displaying token usage trends.
- **Model Usage:** Graph highlighting requests to the model.
- **Latency Trends:** Graph highlighting the latency faced by each trace.
- **Application Level Insights:** Cost distribution, request distribution.
- **Evaluation Level Insights:** Evaluation trend.

These metrics and graphs provide quick insight into:

- Application usage patterns
- Model consumption and cost trends
- Evaluation coverage and quality monitoring
- Overall system health at a glance

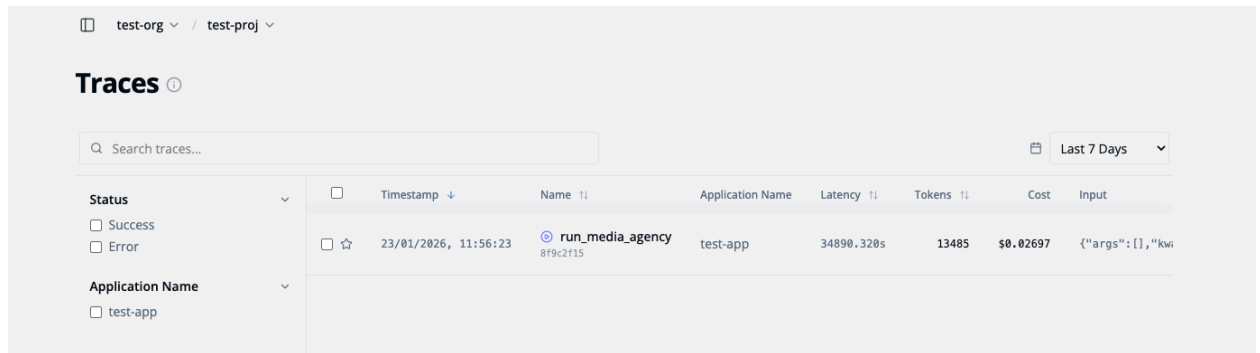
This overview helps users quickly assess performance and identify anomalies without drilling into individual Traces.



6. First Trace Walkthrough (Traces)

After your first successful run, a Trace will appear in the Traces page of the dashboard. This section explains how to explore and understand a single Trace.

1. Navigate to the **Traces** page from the left navigation panel.
2. Click on the most recent Trace to open its detailed view.



A Trace represents one complete execution of your Application and is composed of multiple Observations.

Within the Trace view, you can inspect:

- **Trace Summary:**
Displays high-level information such as trace ID, timestamp, total tokens, latency, and evaluation status.
- **Observation Timeline:**
Shows each step of the execution in sequence, including:
 - Agent calls
 - Tool invocations
 - LLM requests and responses
 - Intermediate processing steps
- **Inputs and Outputs:**
For each Observation, you can view the prompt, response, and structured data exchanged during execution.
- **Metadata:**
Includes model name, parameters, execution duration, and custom tags.

This walkthrough enables you to:

- Debug incorrect behavior in agent workflows
- Understand how tools and models are being used
- Analyze latency and token consumption per step
- Validate evaluation results once metrics are enabled

Note: Each Observation is linked to the same Trace, allowing you to reconstruct the full execution flow from start to finish.

Trace

8f9c2f151ecea66f17f00832d10192d8

Evaluate Trace

EXECUTION TREE

1 ROOTS

run_media_agency

34890ms

planner_agent

2677ms

ChatGroq.invoke

2668ms 1219tok

researcher_agent

7559ms

google_search

500ms

ChatGroq.invoke

7058ms 4414tok

writer_agent

2949ms

ChatGroq.invoke

2947ms 4216tok

editor_agent

11819ms

ChatGroq.invoke

11819ms 2363tok

qc_agent

9876ms

ChatGroq.invoke

9375ms 1273tok

cms_upload

500ms

> run_media_agency

ID: 59557472

Evaluate Node

34890.000s

Tokens 0

Cost \$0.00000

Preview

Log View

INPUT

METADATA

The screenshot displays the LangChain Trace interface. On the left, the 'EXECUTION TREE' shows a sequence of nodes: `run_media_agency` (34890ms), `planner_agent` (2677ms), `ChatGroq.invoke` (2668ms, 1219tok), `researcher_agent` (7559ms), `google_search` (500ms), `ChatGroq.invoke` (7058ms, 4414tok), `writer_agent` (2949ms), `ChatGroq.invoke` (2947ms, 4216tok), `editor_agent` (11819ms), `ChatGroq.invoke` (11819ms, 2363tok), `qc_agent` (9876ms), `ChatGroq.invoke` (9375ms, 1273tok), and `cms_upload` (500ms). The `ChatGroq.invoke` node at 2668ms is selected.

The right panel shows the details for the selected `ChatGroq.invoke` node (ID: 51585072). It includes an 'Evaluate Trace' button and a summary bar showing 2668.000s, 1219 Tokens, and a cost of \$0.00244 using the `openai/gpt-oss-120b` model. Below this, the 'Preview' tab is active, showing the 'INPUT' and 'OUTPUT'.

INPUT:

```
You are a Content Planner. Create a brief outline.  
Create a blog post about AI Agents.
```

OUTPUT:

```
**Blog Post Outline: "AI Agents – The Intelligent Helpers Shaping Tomorrow"**  
  
| Section | Heading (H-Tag) | Core Points / Sub-topics |  
|-----|-----|-----|  
| 1 | **Introduction** (H1) | • Hook: a vivid example of an AI agent in daily life (e.g., a personal finance chatbot) <br>• Quick definition: "software entities that can perceive, reason, and act autonomously on behalf of a user or system." <br>• Why they matter now – advances in LLMs, cloud compute, and APIs. |  
| 2 | **What Exactly Is an AI Agent?** (H2) | • Distinguish from "AI model" and "software bot." <br>• Core components: **Perception**, **Decision-Making**, **Action**. <br>• Types of agents: **Reactive**, **Deliberative**, **Hybrid**. |  
| 3 | **Key Technologies Behind Modern AI Agents** (H2) | • Large Language Models (LLMs) – natural-language understanding & generation. <br>• Retrieval-Augmented Generation (RAG) – grounding responses in up-to-date data. <br>• Reinforcement Learning & Multi-Agent Coordination. <br>• APIs, embeddings, and vector stores for tool use. |  
| 4 | **Common Categories & Real-World Examples** (H2) | **a. Personal Assistants** – Siri, Alexa, Google Assistant (voice, scheduling, smart-home control). <br>**b. Enterprise Agents** – sales-assistant bots, HR onboarding agents, IT help-desk agents. <br>**c. Autonomous Systems** – self-driving car planners, robotic process automation (RPA) bots. <br>**d. Specialized Domain Agents** –
```

6.1 Evaluate Node

The **Evaluate Node** option allows you to evaluate a specific step within a Trace, such as:

- An agent execution
- A tool call
- An individual LLM response

When selected, an evaluation modal is displayed where you can configure the evaluation parameters.

In the evaluation modal, you can provide:

- **Metric:** Select the evaluation metric (for example, Faithfulness, Relevance, or Hallucination).
- **Input Payload / Query:** The input prompt or request used in the selected node.
- **Output / Response:** The model output to be evaluated.
- **Retrieval Context:** Optional supporting context (for RAG-based evaluations).
- **Expected Output (Optional):** Ground truth reference for comparison.

Under **Evaluator Configuration**, you can choose:

- A registered model from the **Model Hub**, or
- A custom configuration by specifying:
 - Provider (for example, OpenAI)
 - LLM API key
 - Model name (for example, gpt-4o)

You can also enable **Trace this evaluation** to log the evaluation run itself as a Trace for further inspection.

6.2 Evaluate Trace

The **Evaluate Trace** option evaluates the entire Trace as a whole instead of an individual node. This mode is intended for **agentic evaluations**, such as:

- Tool selection correctness
- Tool sequence validation
- Agent Routing
- HITL evaluation etc...

When evaluating a full Trace:

- Only agent-level evaluation metrics are available
- The system automatically aggregates all relevant Observations from the Trace
- Results are stored as evaluation scores linked to the Trace

Run Trace Evaluation



Trace Data

Application: test-app

Metric

Faithfulness



Input Payload / Query

```
{
  "args": [
    "latest trends in **Blog Post
    Outline:"
  ]
}
```

Output / Response

Search results for latest trends in **Blog Post Outline:: [Trend A, Trend B, Factor C]

Retrieval Context (Split by double newline)

Kubernetes is an open source container orchestration engine for automating deployment, scaling, and management of containerized applications.

Expected Output (Optional)

Ground truth...

Evaluator Configuration

Registered Model

Custom Config

Select Model

groq-llm (openai/gpt-oss-120b)



Using registered credentials for **groq-llm**.

Tracing

☒ Trace this evaluation

Observix Host

http://localhost:8000

Observix API Key (Ingest)

test-app Key (sk-29mdj...)



PASSED

Score: 1.000

The score is 1.00 because there are no contradictions; the actual output aligns perfectly with the retrieval context. Great job!

Trace

captured: [d14230d38c569b34cb293e2f6f6bb845](#)

Run Trace Evaluation

Trace Data

Metric

Tool Selection

Trace JSON (Required for Observix)

```
{  "spans": [    {      "trace_id": "8f9c2f151ecea66f17f00832d10192d8",      "span_id": "89b54f3591d4319e",      "parent_span_id": null,      "name": "run_media_agency",      "kind": "INTERNAL",      "start_time": "2026-01-23T11:56:23.124277",      "end_time": "2026-01-23T11:56:58.014597",      "status_code": "UNSET",      "status_message": null,
```

Workflow Details (Required for Observix)

```
{  "agents": [],  "tools": []}
```

Evaluator Configuration

Registered Model

Custom Config

Provider

OpenAI

LLM API Key

sk-...

Model Name

gpt-4o

Tracing

☒ Trace this evaluation

Observix Host

http://localhost:8000

Observix API Key (Ingest)

test-app Key (sk-29mdj...)

Close

Run Evaluation

Note: agents, and tool will be automatically populated

Evaluation traces will be tagged as "eval"

7. Model Hub

The Model Hub is a centralized registry where users can register and manage LLM models that are used for running evaluations across Traces and Applications. It enables consistent, reusable, and configurable evaluator models for both manual and automatic evaluations.

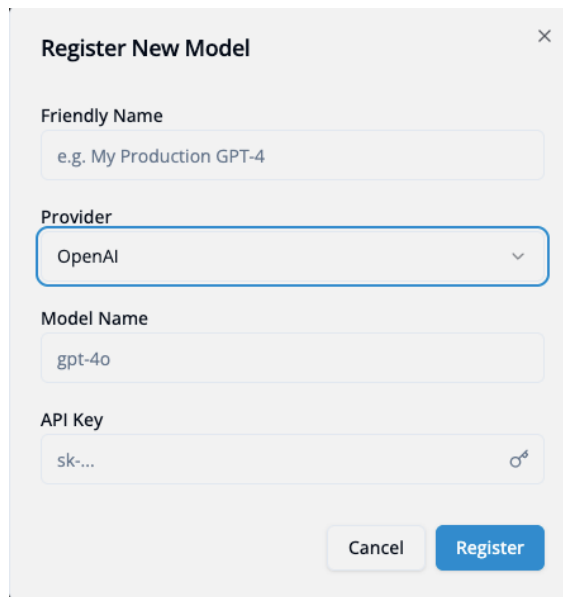
The Model Hub allows users to:

- Register multiple LLM providers (such as OpenAI, Azure OpenAI, Groq, or custom models)
- Manage API keys and model configurations securely
- Reuse registered models across different metrics and Projects

7.1 Registering a Model

To register a new model:

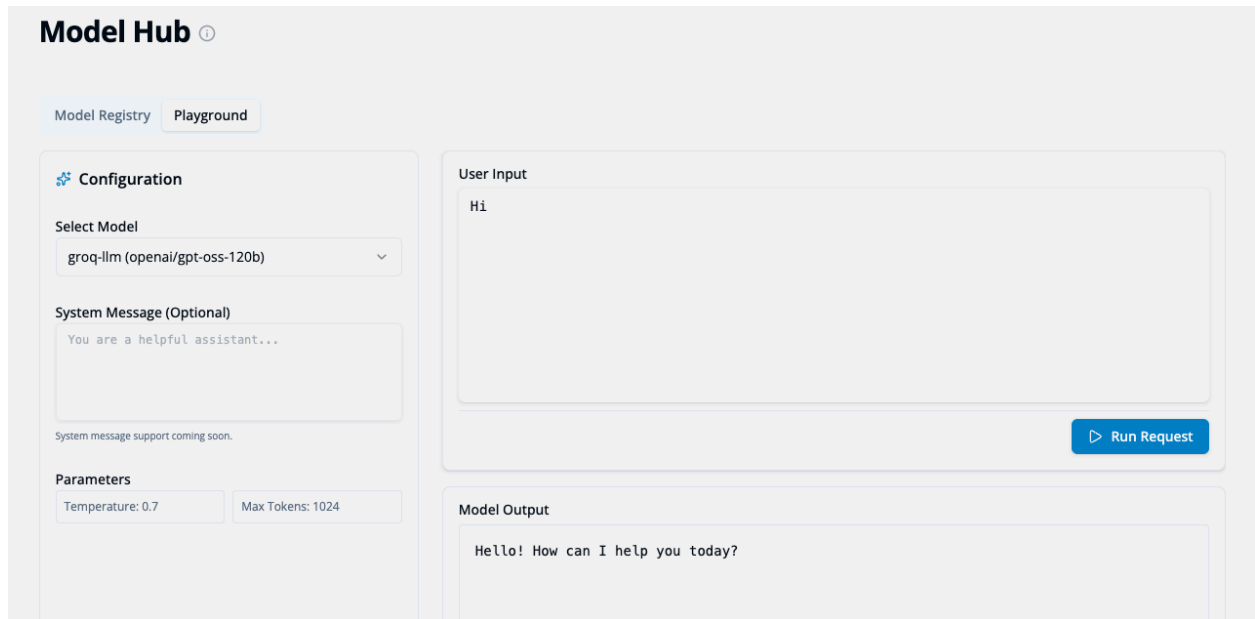
1. Navigate to the Model Hub from the dashboard.
2. Click Add Model.
3. Select the Provider (Currently we only support OpenAI, Azure OpenAI, Groq).
4. Enter the required credentials:
 - API Key
 - Model Name
5. Click Register to save the model.

A screenshot of a 'Register New Model' dialog box. The dialog has a title bar with a close button (X). It contains four input fields: 'Friendly Name' with a placeholder 'e.g. My Production GPT-4', 'Provider' with a dropdown menu showing 'OpenAI', 'Model Name' with a placeholder 'gpt-4o', and 'API Key' with a placeholder 'sk-...' and a copy icon. At the bottom right, there are two buttons: 'Cancel' and 'Register'.

Once registered, the model becomes available for:

- Model Hub Playground
- Manual evaluations (Evaluate Node / Evaluate Trace)
- Auto Evaluation workflows
- Metric testing in the Metric Hub playground

In the **Playground**, you can test the newly registered models



8. Metric Hub

The **Metric Hub** is a centralized catalog of all available evaluation metrics in the platform. It provides detailed metric definitions, configuration guidance, and an interactive playground to test and validate metrics before applying them to real Traces.

The Metric Hub enables users to:

- Explore supported evaluation metrics
- Understand how each metric works and what it measures
- Configure metric parameters
- Test metrics on sample inputs and outputs
- Validate evaluation behavior before enabling Auto Evaluation
- How to integrate your agentic application with that specific metric using our python sdk

Metrics Hub ⓘ

+ Create Metric

SOURCE

All Metrics

SDK Metrics

Custom Metrics

TAGS

preset

custom

rag

safety

agents

tools

Search by name or tag...

Metric Details	Type	Required Inputs	Tags
Answer Relevancy Measures if the response answers the specific question asked.	RAG Evaluation	query response context	preset rag
Faithfulness Measures if the answer is derived faithfully from the retrieved context.	RAG Evaluation	query response context	preset rag
Contextual Precision Measures the precision of the retrieved context.	RAG Evaluation	query response context	preset rag
Contextual Recall Measures the recall of the retrieved context.	RAG Evaluation	query response context	preset rag
Hallucination Determines if the LLM output contains hallucinations based on context.	LLM	query response context	preset safety
Task Completion Measures if the LLM successfully completes the task.	Agentic AI	query response context	preset agents
Tool Correctness Measures if the LLM correctly uses tools.	Agentic AI	query response context	preset agents tools



Answer Relevancy

RAG Evaluation

Measures if the response answers the specific question asked.

Definition & Usage

Run Evaluation

<> Python SDK Usage

Copy

```
from observix.evaluation import AnswerRelevancyEvaluator

llm_kwargs = {
    'api_key': api_key,
    'azure_endpoint': azure_endpoint,
    'api_version': api_version,
    'deployment_name': deployment_name
}

evaluator = AnswerRelevancyEvaluator(provider='openai', model='gpt-4o',
**llm_kwargs)
result = evaluator.evaluate(input_query='', output='', context=[]),
expected='')
```

Tags

preset rag

How it works

This metric uses a dedicated LLM to grade the output based on predefined criteria. It analyzes the semantic meaning rather than just keywords.

ABOUT THIS METRIC

This metric is provided by **Observix**. It is designed to evaluate RAG Evaluation interactions at the Trace level.

Last Updated
-

Complexity
Medium

8.1 Metrics

- **Answer Relevancy:** Measures whether the response directly addresses the specific question asked.
- **Faithfulness:** Measures whether the response is faithfully derived from the provided retrieval context.
- **Contextual Precision:** Measures how much of the retrieved context is relevant to answering the query.
- **Contextual Recall:** Measures how completely the retrieved context covers the information needed to answer the query.
- **Hallucination:** Determines whether the LLM output contains information not supported by the provided context.
- **Task Completion:** Measures whether the agent successfully completes the assigned task.
- **Tool Correctness:** Measures whether the agent correctly uses tools to solve the given task.
- **Toxicity:** Measures whether the LLM output contains toxic, harmful, or unsafe content.
- **Bias:** Measures whether the LLM output exhibits biased or unfair behavior.
- **Tool Selection:** Measures whether the agent selected the most appropriate tool for the task.
- **Tool Input Structure:** Measures whether the tool input arguments follow the required schema and format.
- **Tool Sequence:** Measures whether the sequence of tool calls is logical and efficient.
- **Agent Routing:** Measures whether the request was routed to the correct specialized agent.
- **Human-in-the-Loop:** Measures whether the agent correctly identifies when human intervention is required.
- **Workflow Completion:** Measures whether the overall multi-step workflow was completed successfully.
- **Custom Metric:** Evaluates responses using a user-defined custom prompt or evaluation logic.
- **Accuracy:** Measures how correct the response is compared to a ground truth or expected output.

8.2 Metric Playground

The **Metric Playground** allows users to interactively test metrics with custom inputs before applying them to production data.

In the playground, users can provide:

- **Input Payload / Query:** The prompt or request

- **Output / Response:** The generated model output
- **Retrieval Context:** Supporting documents or knowledge base content (optional)
- **Expected Output:** Ground truth reference (optional)

Users can then:

1. Select a metric from the Metric Hub
2. Choose an evaluator model (from Model Hub or custom configuration)
3. Run the metric in real time
4. View the resulting score and explanation

This helps users:

- Validate metric behavior
- Tune prompts for agentic evaluations
- Compare evaluator models
- Debug unexpected evaluation results

←

Answer Relevancy

RAG Evaluation

Measures if the response answers the specific question asked.

Definition & Usage

Run Evaluation

Input / Query

What is the capital of France?

Context (Optional - Line separated)

Paris is the capital and most populous city of France.

Output / Response

The capital of France is Paris.

Expected Output (Optional)

Paris

Run Evaluation

Configuration

Registered Model

Custom Config

Select Model

groq-llm (openai/gpt-oss-120b)

Using registered credentials for groq-llm.

☐ Observe Trace

Evaluation Result

Score

1

REASONING

"The score is 1.00 because the response directly answered the question with no irrelevant content."

9. Evaluations

The Evaluation module enables users to measure the quality and performance of their Applications using predefined and custom metrics. It provides visibility into evaluation results through dashboards, detailed evaluation runs, and automated evaluation workflows.

9.1 Evaluation Dashboard

The **Evaluation Dashboard** provides a centralized view of all evaluation results across Traces and Applications.

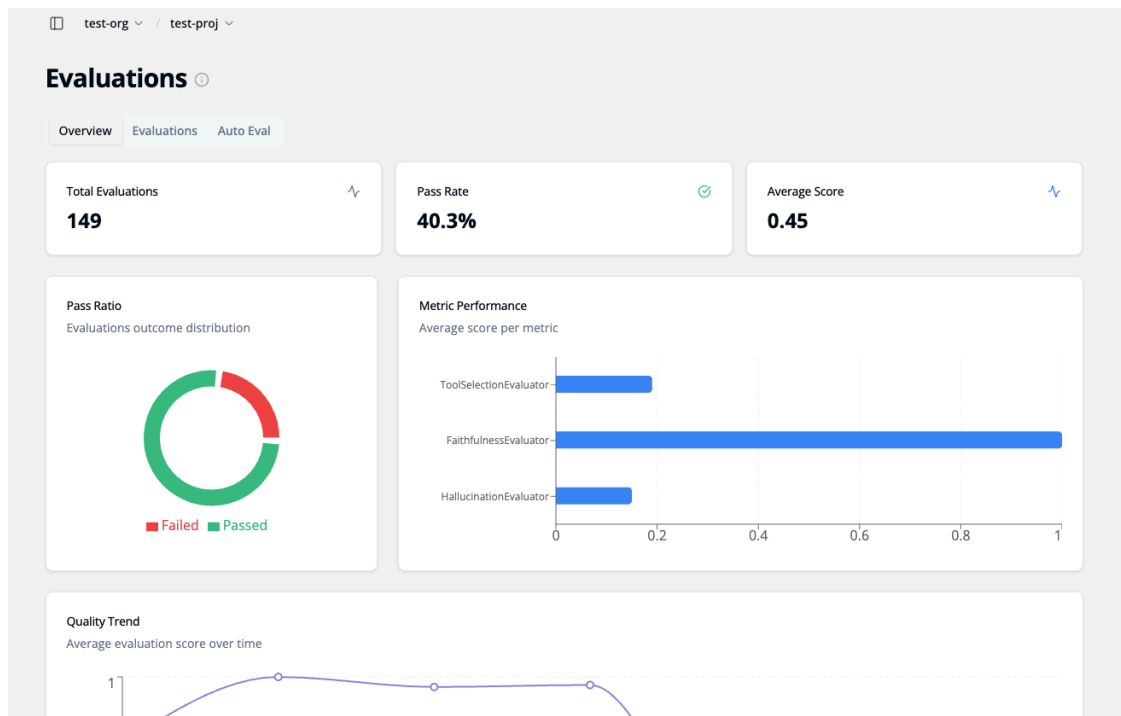
It displays:

- Total number of evaluations executed
- Average scores per metric
- Pass/fail distribution based on configured thresholds
- Trends over time for each metric

Users can:

- Identify quality regressions and anomalies
- Monitor overall system performance at a glance

The dashboard helps teams quickly understand how their agent workflows are performing against defined quality standards.



9.2 Evaluation Runs

An **Evaluation Run** represents a single execution of a metric on a Trace or a specific node within a Trace.

Evaluation runs can be triggered in two ways:

- **Manual Evaluation:** Using *Evaluate Node* or *Evaluate Trace* from the Trace detail page
- **Automatic Evaluation:** Through Auto Evaluation rules

Each Evaluation Run records:

- Metric name
- Evaluated input and output
- Evaluator model used
- Score and result (pass/fail if applicable)
- Timestamp and execution metadata
- Trace ID

Users can inspect individual evaluation runs to:

- Debug incorrect outputs
- Compare results across different models
- Understand why a particular score was assigned
- Trace evaluation outcomes back to the original Trace or Observation

Evaluations ⓘ

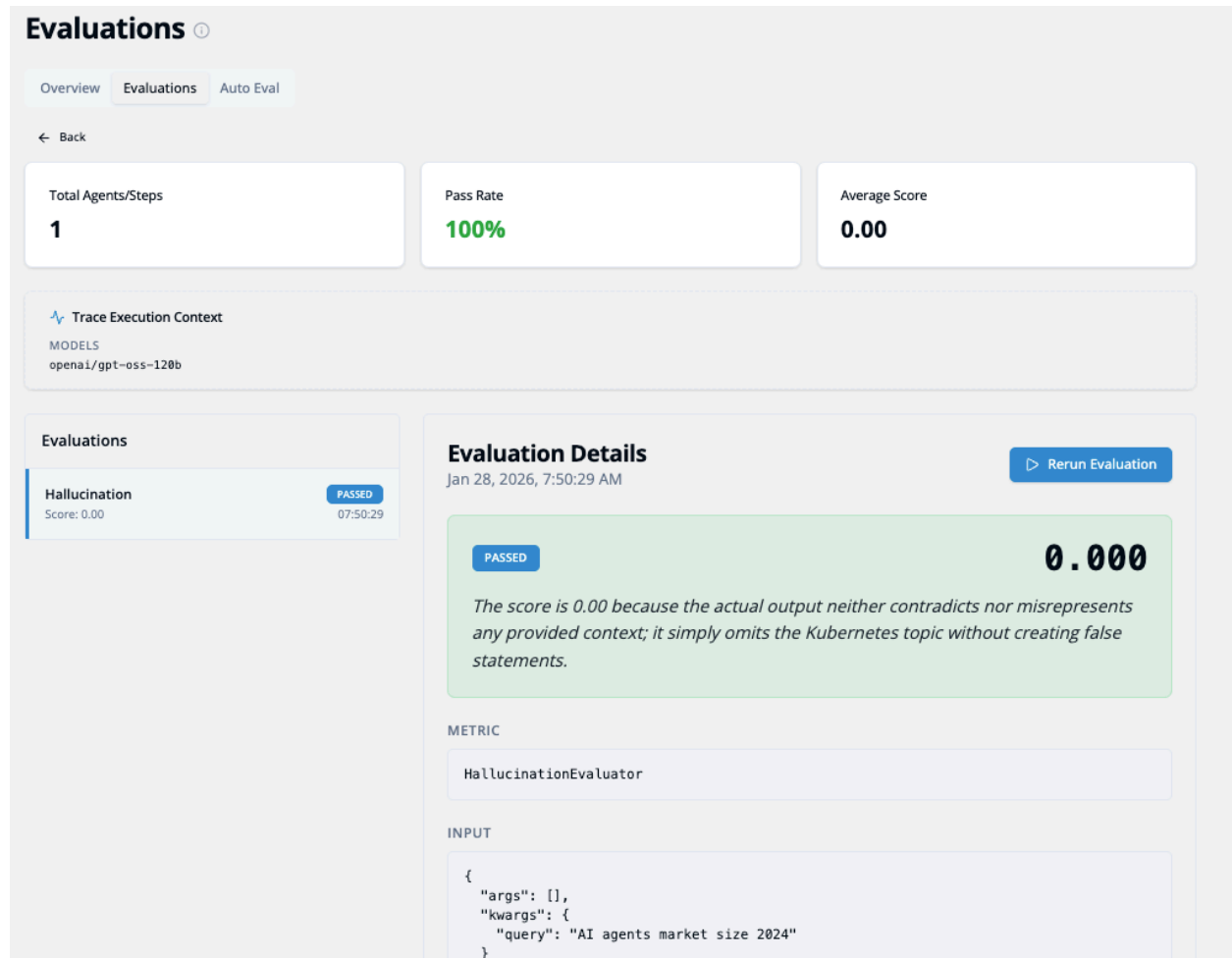
Overview

Evaluations

Auto Eval

Refresh

Timestamp	Application	Trace ID	Status	Pass Status	Avg. Score	Eval Count	Actions
Jan 28, 07:50	test-app-1	12307d65... 🔗	COMPLETED	ALL PASSED	0.00	1	View Details
Jan 27, 08:28	test-app-1	1af3bb1c... 🔗	COMPLETED	ISSUES FOUND	0.20	5	View Details
Jan 27, 08:25	test-app-1	ff1b0908... 🔗	COMPLETED	ALL PASSED	0.00	5	View Details
Jan 27, 08:01	test-app-1	91d12b8a... 🔗	COMPLETED	ALL PASSED	0.00	5	View Details
Jan 27, 07:58	test-app-1	c9586f55... 🔗	COMPLETED	ISSUES FOUND	0.20	5	View Details



9.3 Auto Evaluation

Auto Evaluation allows users to automatically run selected evaluation metrics on every tracked Trace without manual intervention.

Users can configure Auto Evaluation by:

- Selecting metrics from the Metric Hub
- Choosing an evaluator model from the Model Hub
- Defining rules at the Project or Application level

Once enabled:

- Each new Trace triggers the configured evaluation metrics
- Scores are stored and displayed in the Evaluation Dashboard
- Quality trends are continuously monitored in real time

Auto Evaluation is commonly used for:

- Continuous quality monitoring
- Hallucination detection
- Tool selection and sequence validation
- Regression testing of agent workflows

This enables teams to scale evaluation across production workloads and ensure consistent quality over time.

Create Auto-Eval Rule

Rule Name

Hallucination and Answer Relevancy Check

Application

test-app-1

Evaluation Metrics (Select multiple)

×

Answer Relevancy

☐ Faithfulness

☐ Contextual Precision

☐ Contextual Recall

×

Hallucination

☐ Task Completion

☐ Tool Correctness

☐ Toxicity

☐ Bias

Hallucination

Answer Relevancy

Model Configuration

groq-openai (langchain - openai/gpt-oss-120b)

Inputs Override & Model Config (JSON)

Load Template

```
{  "provider": "langchain",  "model": "openai/gpt-oss-120b"}  
```

By default, input/output/context are extracted from the trace. Use this to override inputs or set model config.

Create Rule